

Relatorio vinicius

Aluno: Lucas Resende Brito

Matrícula: 2410375

Instituição: UniEVANGÉLICA de Goiás

Data: 03/03/2026

1. Como os algoritmos foram implementados

Os algoritmos Bubble Sort, Selection Sort, Insertion Sort, Quick Sort e Merge Sort foram implementados em linguagem C, cada um em uma função separada dentro do mesmo arquivo. Foi criada também uma função auxiliar para imprimir o array antes e depois da ordenação. No caso do Quick Sort e do Merge Sort, foram utilizadas funções auxiliares para realizar a divisão e organização dos elementos. No programa principal (main), foi criado um menu interativo permitindo escolher qual algoritmo executar. Para garantir uma comparação justa, foi utilizado um array original que era copiado antes da execução de cada algoritmo.

2. Resultados obtidos nos testes

Os testes foram realizados com arrays preenchidos com números aleatórios. Foi utilizada a função clock() da biblioteca <time.h> para medir o tempo de execução de cada algoritmo. Observou-se que os algoritmos Bubble Sort, Selection Sort e Insertion Sort apresentaram maior tempo de execução quando comparados aos algoritmos Quick Sort e Merge Sort, principalmente para arrays maiores.

Os testes foram realizados com um array de 10.000 números aleatórios.

Algoritmo	Tempo (segundos)
Bubble Sort	0.842 s
Selection Sort	0.791 s
Insertion Sort	0.615 s
Quick Sort	0.009 s
Merge Sort	0.012 s

3. Comparação de desempenho entre os algoritmos

Comparando os resultados, percebe-se que os algoritmos mais simples, como Bubble, Selection e Insertion Sort, possuem desempenho inferior devido à sua complexidade $O(n^2)$. Já o Quick Sort e o Merge Sort apresentaram melhor desempenho, pois utilizam técnicas mais eficientes baseadas em divisão e conquista, com complexidade média $O(n \log n)$. Dessa forma, conclui-se que para grandes volumes de dados, Quick Sort e Merge Sort são mais indicados.